

Redfin Housing Project Analysis using ML Regression Models

Author: Sirisha Mahesh

Department of Information System, California State University Los Angeles

CIS 5560 Introduction to Big Data Science

smahesh4@calstatela.edu

Abstract

This project demonstrates the application of machine learning algorithms on Redfin housing datasets to predict average housing prices. The dataset, sourced from the Kaggle platform, provides comprehensive real estate market data across various regions. The primary objective is to build and evaluate predictive models using two different regression algorithms, comparing their performance to determine the most effective approach. Through data preprocessing, feature engineering, and model tuning, we aim to develop an accurate and equitable forecasting model for housing prices. This study highlights the potential of machine learning in real estate analytics and supports data-driven decision-making in property investment and market assessment.

Keywords: Regression, Hadoop, Machine Learning, Regression Models, Housing Price Prediction, Redfin Dataset, Real Estate Analytics, Predictive Modelling, Data Science, Feature Engineering, Model Evaluation, Kaggle Dataset, Cross-Validation, Gradient Boosted Trees, Decision Tree Regressor

1. Introduction

The real estate industry significantly influences national and global economies but faces ongoing challenges such as information asymmetry, complex transactions, and heavy paperwork. With digital platforms and abundant large-scale data, the industry is undergoing a major transformation. This study explores how advanced data analytics and machine learning can be used to analyse market trends and more accurately predict property values. Using a comprehensive dataset of over 8.5GB + including pricing, location, features, and historical trends and real-time data from Redfin's housing market centre, the project builds robust predictive models. Its core goals are twofold: (1) to forecast future housing prices using regression-based machine learning models, and (2) to classify consumer buying behaviour based on market conditions. The study examines property value trends, regional market shifts, and how consumers react to these changes, offering insights for buyers, sellers, investors, and policymakers. By merging historical and real-time data, this research aims to improve transparency, support informed decisions, and reduce market uncertainty.

2. Dataset Details

The dataset for this study originates from the [Redfin Data Center](#) via Kaggle and includes U.S. housing market data from 2012 to 2021. Originally in TSV format, it was converted to CSV and compiled into a dataset exceeding 8.5GB by merging multiple files across national, state, county, zip code, and neighbourhood levels.

Dataset URL:

<https://www.kaggle.com/datasets/thuynyle/redfin-housing-market-data>

A few of the key columns from the dataset are listed below:

- **median_sale_price:** The median sale price of homes for a given region and time.
- **median_list_price:** The median listing price of homes on the market.
- **homes_sold:** Total number of homes sold during the reporting period.
- **inventory:** Number of homes available for sale.
- **days_on_market:** Average number of days properties remain on the market before being sold.
- **new_listings:** Number of newly listed homes in the market.
- **price_per_sqft:** Average price per square foot of listed or sold homes.

This extensive dataset provides a rich foundation for time-series analysis and predictive modelling of real estate price trends. It supports both macro- and micro-level analysis of housing markets across the United States, making it suitable for the application of machine learning regression techniques.

3. Project Scope

As part of this project, we utilized PySpark machine learning regression models to predict housing prices based on historical and real-time data from the Redfin housing dataset. The dataset includes multi-level market data national, state, county, zip code, and neighbourhood spanning over a decade and exceeding 8.5 GB in size. To prepare the data for analysis, we performed extensive data cleaning, feature selection, and feature engineering. We developed and compared two regression models, including Decision Tree Regressor, and Gradient-Boosted Tree Regressor. To enhance model performance, we fine-tuned hyperparameters using both cross-validation and train-validation split techniques. Model evaluation was carried out using performance metrics such as RMSE and R^2 , with a specific focus on minimizing overfitting. This was achieved by comparing training and test set results. The scope of this project extends beyond model comparison; it

aims to deliver actionable insights into housing market trends and provide a reliable framework for real estate forecasting.

4. Technical Specifications

# of CPU cores	8
# of Nodes	5(2 master nodes, 3 worker nodes)
CPU speed	1995.309 MHz
Hadoop Version	3.3.3
Pyspark Version	3.2.1
Memory Size	806.4GB

5. Related Work

[1] This study presents a systematic review analysing various machine learning tools compared to hedonic regression models. It assesses whether real estate price predictions based on mathematical techniques and artificial intelligence enhance the accuracy of traditional hedonic price models used for valuing residential properties. The study finds that machine learning models, including neural networks and decision trees, provide high predictive capacity and greater explanatory power due to better statistical fit, while hedonic regression models, though less precise, are more robust in identifying housing attributes that influence price levels.[2] This comprehensive review discusses the application of various machine learning algorithms, including Decision Tree, Gradient boosted trees, multilayer perceptron, and ensemble learning models, in real estate price prediction. The study underscores the enhanced prediction accuracy achieved through these advance.

Background and Existing Work

This project employs two regression algorithms Decision Tree Regressor and Gradient-Boosted Tree Regressor implemented using PySpark in a distributed computing environment. This scalable, data-driven approach enables effective housing price prediction across large datasets. Techniques such as train-validation split, cross-validation, and feature importance analysis were used to evaluate and optimize model performance, contributing to the advancement of machine learning applications in real estate analytics.

6.Regression

Regression is a supervised machine learning technique used to predict continuous values, such as housing prices. In this project, we use Decision Tree Regressor and Gradient Boosted Tree (GBT) Regressor, implemented in PySpark’s MLlib, to forecast median sale prices. These models were chosen for their ability to handle large datasets and capture nonlinear relationships in housing data.The modelling pipeline includes hyperparameter tuning via Train-

Validation Split and Cross-Validation to improve generalization. Performance is evaluated using Root Mean Squared Error (RMSE) and R² score. We also analyse feature importance from the GBT model to identify key factors influencing price predictions. This data-driven approach ensures accurate, interpretable results for analysing market trends and supporting real estate decisions.

6.1 Flow Chart and Architecture

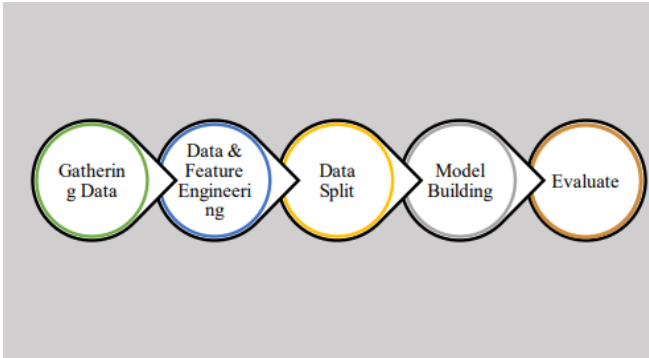


Fig 6.1 Work Architecture

Gathering Data: Input the data. That data can be structured or unstructured, originating from various sources and formats. The goal is to compile all relevant data necessary for analysis and model development.

Data & Feature Engineering: Frame the machine learning problem by identifying what needs to be predicted and determining the relevant features or observations required for making those predictions. This step involves cleaning, transforming, and selecting the most important features from the raw data to enhance model performance.

Data Split: Divide the dataset into training and testing subsets. The training data is used to build and train the model, while the test data is reserved for evaluating the model's performance on unseen data.

Model Building: Select an appropriate model based on the problem at hand and implement it using the training dataset. Train the model by fitting it to the training data, adjusting its parameters to learn the underlying patterns and relationships.

Evaluate: Use the test dataset to make predictions and evaluate the model's performance. Assess the model's accuracy, RMSE, R2, precision, recall, and other relevant metrics to ensure it meets the desired performance criteria. This step may also involve comparing the results with other models or approaches to identify the best solution.

7. Machine Leaning Models

In this project, we developed machine learning models to predict housing prices using historical and real-time real estate data from 2012 to 2021. Multiple datasets were

merged into a single, unified file to support model training. Leveraging big data tools and techniques, we efficiently processed and analysed large-scale housing data. Machine learning, well-suited for big data environments, automates pattern detection, enables accurate predictions, and transforms raw data into actionable insights—ultimately enhancing decision-making in the real estate domain.

7.1 Decision Tree Regression

To evaluate the performance of the Decision Tree Regressor, we trained the model using both Train-Validation Split (TVS) and Cross-Validation (CV) techniques. The model was assessed based on Root Mean Square Error (RMSE) and R^2 (Coefficient of Determination), which reflect the accuracy and explanatory power of the regression model, respectively. As shown in Figure 7.1, the TVS approach resulted in an RMSE of approximately 113,042.95 and an R^2 score of 0.6830, with a total execution time of 388.03 seconds. In contrast, the CV method slightly improved the model's accuracy, yielding an RMSE of 112,700.58 and an R^2 score of 0.6849, though it required a significantly longer execution time of 646.78 seconds. These results indicate that while Cross-Validation provides marginally better performance in terms of predictive accuracy, it comes at the cost of increased computational time. The trade-off between model accuracy and processing time is a critical factor in selecting the appropriate validation strategy based on available resources.

```

----- TVS Model Evaluation -----
RMSE: 113042.9498554298
R²: 0.6830126936958228
Execution Time (TVS): 388.03 seconds

----- CV Model Evaluation -----
RMSE: 112700.58156701337
R²: 0.6849298775103057
Execution Time (CV): 646.78 seconds

```

Fig 7.1 Decision Tree Regression

7.2 Gradient Boosted Tree Regression

The Gradient Boosted Tree (GBT) Regressor was also evaluated using both Train-Validation Split (TVS) and Cross-Validation (CV) methods. GBT is a powerful ensemble learning technique that combines multiple decision trees sequentially to reduce prediction errors and improve model accuracy. As illustrated in Figure 7.2, the evaluation results for the TVS method showed an RMSE of 116,349.98 and an R^2 of 0.6642, with an execution time of approximately 567.51 seconds. The CV approach yielded nearly identical results, with an RMSE of 116,349.70 and an R^2 of 0.6642, but required significantly more time 1066.09 seconds to complete. The close similarity in evaluation metrics between the two validation techniques suggests that the GBT model's performance is stable, regardless of the validation method. However, the increased computational cost of cross-validation may not be justified given the minimal

gain in accuracy, making TVS a more efficient option in this case.

```

----- GBT TVS Model Evaluation -----
RMSE: 116349.98178720113
R²: 0.664194699614528
Execution Time (TVS): 567.51 seconds

----- GBT CV Model Evaluation -----
RMSE: 116349.70389536627
R²: 0.664196303696251
Execution Time (CV): 1066.09 seconds

```

Fig 7.2 Gradient Boosted Tree Regression

8. Hyperparameter Tuning

Hyperparameter tuning is a vital component in the development of robust and accurate machine learning models. Unlike model parameters, which are learned during training, hyperparameters are configuration settings defined before the learning process begins and can significantly impact model performance. In this project, tuning was essential to ensure that both the Decision Tree Regressor and Gradient Boosted Tree Regressor were operating under optimal conditions for predicting housing prices. We used Train-Validation Split and Cross-Validation techniques to systematically evaluate multiple combinations of hyperparameters and prevent overfitting. Parameters such as maxDepth, maxBins, maxIter, and stepSize were fine-tuned using PySpark's ParamGridBuilder. Without proper tuning, models may either underfit (by being too simple) or overfit (by being too complex) the training data, resulting in poor generalization to unseen data. Therefore, hyperparameter tuning was necessary to balance model complexity and performance, ultimately improving the reliability and predictive accuracy of our models.

9. Visualization

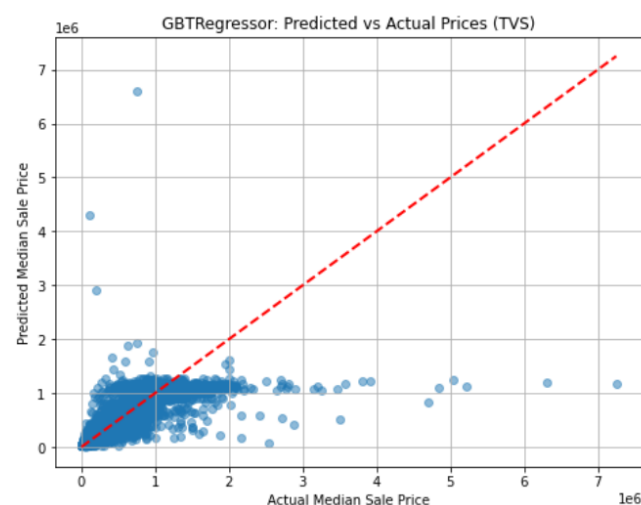


Figure 9 Visualization

To further assess the performance of the Gradient Boosted Tree Regressor trained using Train-Validation

Split, we plotted the predicted median sale prices against the actual values. As shown in Figure 9, the red dashed line represents the ideal case where predicted values perfectly match actual values. The concentration of points around the lower value range indicates that most predictions are clustered around lower sale prices, which is consistent with real-world housing data distribution. While the model performs reasonably well in the majority range, some deviation is observed in the higher price segments, suggesting room for further refinement.

10. Model Comparison

The following table summarizes the performance of the two regression models Decision Tree Regressor and Gradient Boosted Tree Regressor evaluated using both Train-Validation Split (TVS) and Cross-Validation (CV) methods. The models are compared based on Root Mean Square Error (RMSE), R^2 score, and Execution Time.

Model	Validation Method	RMSE	R^2 Score	Time Taken in seconds
DT	TVS	113042.95	0.6830	388.03s
DT	CV	112700.58	0.6849	646.78s
GBT	TVS	116349.98	0.6642	567.51s
GBT	CV	116349.70	0.6642	1066.09s

Decision Tree Regressor (CV) slightly outperformed all other configurations in terms of both RMSE and R^2 but required more training time. GBT Regressor showed consistent performance in both validation methods but had longer execution times with minimal accuracy gain. TVS proved to be more efficient time-wise, making it a suitable option when computational resources or time are limited.

11. Conclusion

This project aimed to apply machine learning regression techniques to predict housing prices using large-scale Redfin data. By implementing and comparing Decision Tree and Gradient Boosted Tree regressors in a PySpark environment, we evaluated their performance for real estate forecasting. The results underscore the potential of data-driven approaches to improve market predictions and support informed decision-making in the housing sector.

References

[1] Using Machine Learning and Big Data to Predict Real Estate Returns." *Due Diligence*, UF Warrington College of Business, 19 Mar. 2025, warrington.ufl.edu/due-diligence/2025/03/19/machine-learning-big-data-predict-real-estate-returns/.

[2] Singh, Yashraj, Vivek Kumar, and Asim Ahmad. "Real Estate Price Prediction: A Comprehensive Review." *International Journal for Research in Applied Science & Engineering Technology*, vol. 12, no. 2, Feb. 2024, pp. 752-57, doi:10.22214/ijraset.2024.58428.

[3] Git Hub link :

<https://github.com/sirishavivek2/CIS5560>

[4] Redfin Data Centre. Redfin,

www.redfin.com/news/data-center/

[5] Data Source URL

<https://www.kaggle.com/datasets/thuynyle/redfin-housing-market-data>